

“Iot Based Smart Storage locker System”

Approach 1: Android Application

Project Overview

This document provides a step-by-step guide to build a **Smart Locker System** that allows users to book a locker via an Android app, pay online, and receive an OTP to open the locker. The system also includes an admin panel for remote control and an ESP32-based hardware controller.

The project consists of three main components:

- **Backend** – Node.js server with SQLite database, REST APIs, Razorpay integration, MQTT unlock commands, and a real-time admin dashboard.
- **Android Application** – Kotlin/Jetpack Compose app for user registration, location selection, locker booking, payment, OTP verification, and admin controls.
- **ESP32 Hardware** – Microcontroller connected to relays and solenoid locks, controlled via MQTT.

All components communicate over a local network (or the internet) as shown:

[Android App] ⇄ HTTP ⇄ [Backend Server] → MQTT → [ESP32] → [Relays & Solenoid Locks]

Table of Contents

1. Prerequisites & Software Download Links
2. Backend Setup (Node.js + SQLite)
3. Android Application Setup (Kotlin + Jetpack Compose)
4. ESP32 Hardware Setup
5. Running the Complete System
6. Admin Dashboard & Admin Login
7. Troubleshooting Common Issues
8. Customization Tips

1. Prerequisites & Software Download Links

All required tools and their official download links are listed below. Install them before proceeding.

Tool / Library	Purpose	Download Link
Node.js (v16 or later)	Backend runtime	Download Node.js
Visual Studio Code (or any editor)	Code writing	Download VS Code
Android Studio (latest stable)	Build the Android app	Download Android Studio
Arduino IDE (2.x)	Program the ESP32	Download Arduino IDE
Git (optional, for version control)	Clone repositories	Download Git
Razorpay Test Account	Payment gateway credentials	Sign up / Log in
ESP32 Board Support URL (for Arduino IDE)	Install ESP32 support	https://dl.espressif.com/dl/package_esp32_index.json
PubSubClient (MQTT)	ESP32 MQTT communication	GitHub Repository / Installed via Arduino Library Manager
ArduinoJson	JSON parsing on ESP32	Official Website / Installed via Arduino Library Manager

Hardware components (purchase from any electronics store):

- ESP32 development board (e.g., DOIT ESP32 DEVKIT V1)
- 4-channel 5V relay module
- 4 × 12V solenoid locks
- 12V DC power supply (at least 4A)
- Jumper wires, breadboard

2. Backend Setup (Node.js + SQLite)

Step 2.1: Create the Project Folder

Open a terminal (Command Prompt / PowerShell on Windows, Terminal on Mac/Linux) and run:

```
mkdir locker-backend
```

```
cd locker-backend
```

```
npm init -y
```

This creates a package.json file.

Step 2.2: Install Backend Dependencies

Run the following single command to install all required Node.js packages:

```
npm install express cors razorpay mqtt sequelize sqlite3 bcryptjs jsonwebtoken socket.io
```

Packages explained:

- express – web framework for building APIs
- cors – allows cross-origin requests (needed for Android)
- razorpay – official Razorpay SDK
- mqtt – MQTT client to communicate with the ESP32
- sequelize + sqlite3 – ORM and SQLite driver for database
- bcryptjs – password hashing
- jsonwebtoken – JWT authentication
- socket.io – real-time communication for the admin dashboard

No separate downloads are needed – npm fetches everything automatically.

Step 2.3: Write the Backend Code

- In your locker-backend folder, create a file named index.js.
- Copy and paste the **provided backend code** into this file.

Step 2.4: Configure the Backend

Before running, open index.js and modify these settings:

- **Razorpay Secret:** Find the line `key_secret: "xxxxxxxxxxxx"`. Replace it with your actual Razorpay key secret (get it from [Razorpay Dashboard](#)).
- **Force Reset Flag:** At the very top of the file, find `const FORCE_RESET = true;`. Leave it as true for the first run only. This will rebuild the database to avoid any schema errors.
- **JWT Secret:** You can keep the default or generate a new strong random string.
- **Admin Credentials:** The default admin is admin / admin123. You can change the password by modifying the seeding block.

Step 2.5: Start the Server

In the terminal, execute:

```
node index.js
```

You should see colored logs showing:

- ✓ Database connected
- ✓ MQTT connected
- ✓ Default admin created
- 🚀 Server running on http://localhost:3000

If you encounter a "**Validation error**" at startup, stop the server (Ctrl+C), delete the database.sqlite file that was created in the folder, and restart. The FORCE_RESET = true flag will handle everything automatically.

Step 2.6: Verify the Backend

- Open a web browser and go to http://localhost:3000/admin. You will see the admin dashboard with a location selector and a table of lockers (all showing "AVAILABLE").
- In the terminal where the server is running, type status and press Enter. A detailed table of all locker states will appear.

3. Android Application Setup (Kotlin + Jetpack Compose)

Step 3.1: Install Android Studio

Download and install Android Studio from the official link above. During installation, ensure the Android SDK and a system image for an emulator are selected.

Step 3.2: Create a New Project

1. Open Android Studio and click **New Project**.
2. Select **Empty Activity**.
3. Fill in the fields:
 - **Name:** Smart Locker
 - **Package name:** com.example.smartlocker
 - **Language:** Kotlin
 - **Minimum SDK:** API 24 (Android 7.0)
4. Wait for the project to sync and build.

Step 3.3: Add Required Dependencies

1. In the **Project** view, navigate to app/build.gradle.kts.
2. Delete the existing contents and replace them with the **provided Gradle file**. This file adds the necessary libraries: Jetpack Compose, Material3, Razorpay, OkHttp, [Socket.IO](#), and desugaring.

3. Click **Sync Now** when Android Studio prompts you.

Step 3.4: Configure the Android Manifest

1. In the **Project** view, go to `app/src/main/AndroidManifest.xml`.
2. Replace its entire content with the **provided AndroidManifest.xml**. This manifest:
 - Requests the INTERNET permission.
 - Allows cleartext traffic (needed to connect to a local HTTP server).
 - Registers the Razorpay receiver and activity.

Step 3.5: Insert the Main Activity Code

1. In the **Project** view, navigate to `app/src/main/java/com/example/smartlocker/`.
2. Delete any existing files (like `MainActivity.kt`) and paste the **provided MainActivity.kt** file.
3. This file contains the entire user interface and all business logic (login, location selection, locker grid, payment, OTP dialog, admin panel).

Step 3.6: Set the Backend URL

1. In `MainActivity.kt`, find the line:

```
private val BASE_URL = http://10.87.190.139:3000
```

2. Replace the IP address with your computer's actual local IP address.
 1. **Windows:** Open Command Prompt and type `ipconfig`. Look for the IPv4 Address under your active network adapter.
 2. **Mac/Linux:** Open Terminal and type `ifconfig`. Look for `inet` under `en0` or `wlan0`.
3. For example, if your IP is 192.168.1.100, the line should be:

```
private val BASE_URL = http://192.168.1.100:3000
```

Step 3.7: Build and Run the App

1. Connect your Android device via USB and enable **Developer Options** → **USB Debugging**. Alternatively, start an Android emulator in Android Studio.
2. Click the green **Run** button in Android Studio. The app will compile and launch on your device.

Step 3.8: Test the Complete App Flow

1. **Sign Up** – Enter a name, email, and password to create an account.

2. **Select a Location** – After logging in, choose one of the five locations (e.g., "Metro Station").
 3. **Book a Locker** – Tap an available (green) locker → select a duration → the Razorpay payment sheet will open. Use test card 4111 1111 1111 1111 with any future expiry date and any CVV.
 4. **Receive OTP** – After payment, the app shows the OTP in a dialog.
 5. **Enter OTP** – Type the 4-digit code and tap "Verify". If correct, the locker turns orange and a timer starts counting up.
 6. **Admin Login** – Log out, toggle the **Admin** switch on the login screen. Use admin / admin123 to access the admin panel. You can force-unlock or force-reset any locker.
-

4. ESP32 Hardware Setup

Step 4.1: Assemble the Hardware

Wire the components according to the connection table provided in the source code. Refer to the hardware diagram included with your project files.

Step 4.2: Install Arduino IDE and ESP32 Support

1. Install the Arduino IDE from the link above.
2. Open the IDE, go to **File** → **Preferences** (Windows) / **Arduino IDE** → **Settings** (Mac).
3. In the "Additional Boards Manager URLs" field, paste:
https://dl.espressif.com/dl/package_esp32_index.json
4. Click **OK**.
5. Go to **Tools** → **Board** → **Boards Manager**, search for `esp32`, and install the package by **Espressif Systems**.

Step 4.3: Install Required Arduino Libraries

1. In the Arduino IDE, go to **Sketch** → **Include Library** → **Manage Libraries**.
2. Search for and install these libraries (click "Install" on each):
 - **PubSubClient** (by Nick O'Leary)
 - **ArduinoJson** (by Benoit Blanchon)

Step 4.4: Upload the ESP32 Code

1. Create a new sketch (File → New) and paste the **provided ESP32 code**.
2. Modify the Wi-Fi credentials near the top of the sketch:

```
const char* ssid = "Your_WiFi_Name";  
const char* password = "Your_WiFi_Password";
```

3. Select your board: **Tools** → **Board** → **esp32** → **DOIT ESP32 DEVKIT V1** (or your specific model).
4. Select the correct COM port under **Tools** → **Port**.
5. Click the **Upload** button (right arrow).

Step 4.5: Verify Hardware Communication

1. Open the **Serial Monitor** (Tools → Serial Monitor, baud rate 115200).
2. You should see:
 - Connecting to WiFi...
 -  WiFi connected
 - Connecting to MQTT... connected
3. Whenever an unlock command is sent from the backend (e.g., via the admin panel), the ESP32 will print "Unlocking L1" and activate the corresponding relay for 2 seconds.

5. Running the Complete System

1. Ensure the backend server is running (node index.js).
2. Confirm the ESP32 is powered on and connected to the same Wi-Fi as the backend.
3. Connect your Android device to the same Wi-Fi network.
4. Launch the Android app and perform the full flow.

6. Admin Dashboard & Admin Login

- **Web Dashboard:** <http://<server-ip>:3000/admin> shows live status of all lockers (read-only).
- **Android Admin Panel:** Use the User/Admin toggle on the login screen. After login, you can choose a location and see all lockers with **Force Unlock** and **Force Reset** buttons.

7. Troubleshooting Common Issues

Problem	Solution
Backend startup "Validation error"	Delete database.sqlite, set FORCE_RESET = true, restart, then set back to false.
Android app "Network error"	Verify BASE_URL IP is correct and the device is on the same Wi-Fi.
Razorpay payment fails	Ensure test key and secret are valid; use test card 4111....
ESP32 not unlocking	Check MQTT broker connection in serial monitor; ensure relay wiring matches active-LOW logic.

8. Customization Tips

- To add more locations, modify the locations array in both index.js and MainActivity.kt.
- To change pricing/durations, edit the rateOptions list in the Android app and the corresponding amounts in the backend's create-order route.
- For production, replace the public MQTT broker with a private one (e.g., Mosquitto).

Approach 2: Desktop App

A full-stack smart locker management platform built with Spring Boot + React

Table of Contents

- [Project Overview](#)
- [System Requirements](#)
- [Project Structure](#)
- [Step-by-Step Setup \(Fresh System\)](#)
 - [1. Install Java 17](#)
 - [2. Install Maven](#)
 - [3. Install Node.js](#)
 - [4. Install Git](#)
 - [5. Clone the Repository](#)
 - [6. Configure the Backend](#)
 - [7. Install & Run the Backend](#)
 - [8. Install & Run the Frontend](#)
- [External Services Setup](#)
- [Running the Application](#)
- [Production Build](#)
- [API Overview](#)
- [Common Errors & Fixes](#)
- [Docker \(Optional\)](#)
- [Security Best Practices](#)
- [Recommended .gitignore](#)

Project Overview

Tuckit is a full-stack smart locker management system that allows users to:

-  Acquire a locker and pay via Razorpay
-  Open/close lockers via OTP sent to email
-  Do full or partial pickups of belongings
-  Automatically track overdue lockers with penalty charges

-  Manage user profiles with emergency contact info

Layer	Technology	Port
Backend	Java 17 + Spring Boot 3.2.3	8080
Frontend	React 18 + Vite + TypeScript	3000
Database	MongoDB Atlas (cloud)	—

✔ System Requirements

Before starting, make sure a fresh system has all of the following installed:

Tool	Minimum Version	What It's For
Java JDK	17	Running the Spring Boot backend
Maven	3.6+	Building the Java project
Node.js	18+	Running the React frontend
npm	9+	Installing frontend packages (comes with Node)
Git	2.x	Cloning the repository

How to check if already installed: Open a terminal and run:

```
java -version
```

```
mvn -version
```

```
node -v
```

```
npm -v
```

```
git --version
```

Project Structure

Takkit/

```
|— backend/           ← Spring Boot application
| |— src/
| |   |— main/
| |       |— java/com/tuckit/ ← Java source code
| |       |— resources/
| |       |— application.properties ← ⚠ NOT in git (you create this)
| |       |— application.properties.example ← ✅ Template (copy this)
| |— target/         ← Build output (auto-generated)
|   |— pom.xml       ← Maven dependencies
|
|— client/           ← React + Vite frontend
| |— src/
| |   |— pages/       ← Login, Signup, Dashboard, Profile
| |   |— components/  ← Navbar, Modals, etc.
| |   |— api.ts        ← Axios API client
| |— public/
| |— package.json
| |— vite.config.ts   ← Proxies /api → localhost:8080
|
|— .gitignore
|— README.md
```

Step-by-Step Setup (Fresh System)

Step 1: Install Java 17

Windows / macOS / Linux:

1. Go to: <https://adoptium.net/temurin/releases/>
2. Select: Version 17, your OS, and architecture (x64 for most laptops)
3. Download and run the installer

4.  Verify: Open a new terminal and run:

5. `java -version`

Expected: openjdk version "17.x.x"

💡 On Ubuntu/Debian Linux, you can also run:

`sudo apt update && sudo apt install openjdk-17-jdk -y`

Step 2: Install Maven

Maven is the build tool for the Java backend.

Windows:

1. Go to: <https://maven.apache.org/download.cgi>
2. Download apache-maven-3.9.x-bin.zip
3. Extract to C:\tools\apache-maven-3.9.x
4. Add to PATH: Search "Environment Variables" → Edit Path → Add C:\tools\apache-maven-3.9.x\bin
5. Open a new terminal and verify:
6. `mvn -version`

Expected: Apache Maven 3.9.x

macOS (with Homebrew):

`brew install maven`

Ubuntu/Debian Linux:

`sudo apt install maven -y`

Step 3: Install Node.js

Node.js is required to run and build the React frontend.

1. Go to: <https://nodejs.org/>
2. Download the LTS version (18 or higher)
3. Run the installer — it also installs npm automatically
4.  Verify:
5. `node -v`
6. # Expected: v18.x.x or higher
- 7.
8. `npm -v`

Expected: 9.x.x or higher

Ubuntu/Debian Linux:

```
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
```

```
sudo apt install nodejs -y
```

Step 4: Install Git

1. Go to: <https://git-scm.com/downloads>
2. Download and install for your OS
3.  Verify:
4. `git --version`

Expected: git version 2.x.x

Step 5: Clone the Repository

Open a terminal, navigate to where you want the project, and run:

```
git clone https://github.com/YOUR_USERNAME/tuckit.git
```

```
cd tuckit
```

You will now see the Tackit/ folder with backend/ and client/ inside.

Step 6: Configure the Backend (Secrets)

The backend needs a configuration file that contains private API keys and secrets. This file is intentionally not included in the repository for security reasons.

You must create it manually by following these steps:

6a. Copy the template file

Navigate into the backend resources folder

```
cd backend/src/main/resources
```

Copy the template

```
cp application.properties.example application.properties
```

Windows users: You can also right-click `application.properties.example` in File Explorer → Copy → Paste → Rename to `application.properties`

6b. Fill in your values

Open `application.properties` in any text editor (Notepad, VS Code, etc.) and replace each placeholder:

— MongoDB Atlas

```
spring.data.mongodb.uri=mongodb+srv://<username>:<password>@<cluster>.mongodb.net/<dbnam
e>?retryWrites=true&w=majority
```

```
# — JWT Secret
```

```
app.jwt.secret=REPLACE_WITH_YOUR_BASE64_SECRET_MIN_64_CHARS
```

```
# — Gmail SMTP
```

```
spring.mail.username=your-gmail@gmail.com
```

```
spring.mail.password=xxxx xxxx xxxx xxxx ← Gmail App Password (16 chars)
```

```
# — Razorpay
```

```
app.razorpay.key-id=rzp_test_XXXXXXXXXXXXXXXX
```

```
app.razorpay.key-secret=XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

```
# — CORS
```

```
app.cors.origins=http://localhost:3000
```

See the [External Services section](#) below for how to get each key.

Step 7: Install & Run the Backend

Open a terminal and navigate to the backend folder:

```
# From the project root:
```

```
cd backend
```

```
# Build the project (downloads all Java dependencies, takes ~1-2 min on first run)
```

```
mvn clean install -DskipTests
```

```
# Run the backend server
```

```
java -jar target/tuckit-backend-1.0.0.jar
```

✅ Success output looks like:

Tuckit Smart Locker System — Backend

Running at http://localhost:8080

The backend is now live at: <http://localhost:8080>

Step 8: Install & Run the Frontend

Open a new terminal (keep the backend running in the other one) and navigate to the client folder:

From the project root:

```
cd client
```

Install all frontend dependencies

```
npm install
```

Start the development server

```
npm run dev
```

✅ Success output looks like:

```
VITE v5.x.x ready in 300ms
```

→ Local: <http://localhost:3000/>

Open your browser and go to: <http://localhost:3000>

External Services You Must Configure

1. MongoDB Atlas (Free Database)

MongoDB Atlas is a free cloud database. The app will not start without a valid URI.

1. Go to <https://cloud.mongodb.com> and create a free account
2. Create a Free Cluster (M0 — Shared, no credit card needed)
3. Click "Connect" → "Drivers" → Copy the connection string
4. Create a database user (username + password)
5. Under Network Access → Add 0.0.0.0/0 to allow all IPs (for development)
6. Paste into application.properties:

```
spring.data.mongodb.uri=mongodb+srv://YOUR_USER:YOUR_PASSWORD@cluster0.xxxxxx.mongodb.net/tuckit?retryWrites=true&w=majority
```

⚠️ URL-encode special characters in passwords! Replace @ with %40, # with %23, etc.

2. JWT Secret Key

Generate a secure random Base64 key (minimum 64 characters):

Online: Visit <https://generate.plus/en/base64> → set length to 64 → copy

Terminal (macOS/Linux):

```
openssl rand -base64 64
```

Terminal (Windows PowerShell):

```
[Convert]::ToBase64String((1..48 | ForEach-Object { [byte](Get-Random -Max 256) } ))
```

Paste into application.properties:

```
app.jwt.secret=YOUR_GENERATED_64_CHAR_BASE64_STRING
```

3. Gmail App Password (for OTP emails)

The app sends OTP codes via email. You cannot use your regular Gmail password — you need an App Password.

1. Enable 2-Step Verification on your Google account: <https://myaccount.google.com/security>
2. Go to App Passwords: <https://myaccount.google.com/apppasswords>
3. Select app: "Mail" → Select device: "Other (Custom name)" → give it any name
4. Click Generate → Copy the 16-character password (format: xxxx xxxx xxxx xxxx)
5. Paste into application.properties:
6. `spring.mail.username=your-gmail@gmail.com`

```
spring.mail.password=xxxx xxxx xxxx xxxx
```

4. Razorpay API Keys (for payments)

1. Create a free account at <https://dashboard.razorpay.com>
2. Go to Settings → API Keys → Generate Test Key
3. Copy the Key ID and Key Secret
4. Paste into application.properties:
5. `app.razorpay.key-id=rzp_test_XXXXXXXXXXXXXXXX`

```
app.razorpay.key-secret=XXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

💡 Use test mode keys (prefix: `rzp_test_`) for development. Switch to live keys only when deploying to production.

Test Card for payments:

Field	Value
Card Number	4111 1111 1111 1111
Expiry	Any future date
CVV	Any 3 digits
OTP	123456

▶ Running the Application

Once setup is complete, use two separate terminals:

Terminal 1 — Backend:

```
cd backend
```

```
java -jar target/tuckit-backend-1.0.0.jar
```

Terminal 2 — Frontend:

```
cd client
```

```
npm run dev
```

Then open: <http://localhost:3000> 🎉

The frontend Vite dev server automatically proxies all `/api/*` requests to `http://localhost:8080`, so no additional configuration is needed.

Building for Production

Backend (create a runnable JAR):

```
cd backend
```

```
mvn clean package -DskipTests
```

```
# JAR file created at: backend/target/tuckit-backend-1.0.0.jar
```

```
# Run it on any machine with Java 17:
```

```
java -jar target/tuckit-backend-1.0.0.jar
```

Frontend (create optimized static files):

```
cd client
```

```
npm run build
```

```
# Output created at: client/dist/
```

```
# Serve this folder with any web server (Nginx, Apache, Vercel, Netlify, etc.)
```

API Overview

All backend endpoints are prefixed with /api.

Method	Endpoint	Auth Required	Description
POST	/api/auth/signup	No	Register new user
POST	/api/auth/login	No	Login and get JWT token
GET	/api/auth/me	Yes (JWT)	Get current user profile
PUT	/api/auth/profile	Yes (JWT)	Update user profile
GET	/api/lockers	Yes (JWT)	List all lockers
POST	/api/lockers/acquire	Yes (JWT)	Acquire a locker
POST	/api/lockers/request-otp	Yes (JWT)	Send OTP to email
POST	/api/lockers/verify-otp	Yes (JWT)	Verify OTP, open locker
POST	/api/lockers/pickup	Yes (JWT)	Pickup items
POST	/api/lockers/close	Yes (JWT)	Close locker door
POST	/api/payment/create-order	Yes (JWT)	Create Razorpay order
POST	/api/payment/verify	Yes (JWT)	Verify payment signature

💡 Health check: GET <http://localhost:8080/api/auth/health>

🔧 Common Errors & Fixes

❌ "java: command not found" or "'java' is not recognized"

Fix: Java is not installed or not in PATH.

- Install Java 17 from <https://adoptium.net/>
- On Windows: restart your terminal after installation

❌ "mvn: command not found"

Fix: Maven is not installed or not in PATH.

- See [Step 2](#) above
- On Windows: after adding to PATH, open a new terminal

✗ Backend won't start — application.properties not found

Fix: You haven't created the config file yet.

```
cd backend/src/main/resources
```

```
cp application.properties.example application.properties
```

```
# Now fill in your real values
```

✗ MongoTimeoutException or MongoSocketOpenException

Fix: Your MongoDB URI is wrong or your IP is not whitelisted.

1. Check your spring.data.mongodb.uri in application.properties
2. In MongoDB Atlas → Network Access → Add 0.0.0.0/0
3. Make sure username/password are correct and special chars are URL-encoded

✗ "Invalid email or password" on login (even with correct credentials)

Fix: Old backend JAR may be running. Kill it and restart:

```
# Windows PowerShell:
```

```
Get-Process java | Stop-Process -Force
```

```
java -jar target/tuckit-backend-1.0.0.jar
```

```
# macOS/Linux:
```

```
pkill -f "tuckit-backend"
```

```
java -jar target/tuckit-backend-1.0.0.jar
```

✗ CORS Error when signing up / logging in

Fix: Your frontend URL is not in the CORS allowed list.

Open application.properties and verify:

```
app.cors.origins=http://localhost:3000,http://localhost:5173
```

Then restart the backend.

✗ Email OTP not received

Fix: Gmail SMTP is not configured correctly.

1. Make sure you're using an App Password (not your Gmail password)
2. Check that 2-Step Verification is enabled on your Google account
3. Look for the email in your spam folder

✖ Razorpay checkout window not opening

Fix:

1. Make sure `app.razorpay.key-id` and `app.razorpay.key-secret` are set in `application.properties`
2. Make sure the Razorpay script is loaded in `client/index.html`:

```
<script src="https://checkout.razorpay.com/v1/checkout.js"></script>
```

3. Rebuild and restart the backend after changing the key

✖ npm install fails

Fix: Clear the cache and retry:

```
npm cache clean --force
```

```
npm install
```

If still failing: delete `node_modules` and `package-lock.json`, then retry:

```
cd client
```

```
rm -rf node_modules package-lock.json
```

```
npm install
```

✖ Port 8080 already in use

Windows:

```
netstat -ano | findstr :8080
```

```
taskkill /PID <PID_NUMBER> /F
```

macOS/Linux:

```
lsof -ti:8080 | xargs kill -9
```

 Docker (Optional Portable Setup)

If you have Docker installed, you can run the backend without needing Java or Maven.

Install Docker: <https://docs.docker.com/get-docker/>

Create backend/Dockerfile:

```
FROM eclipse-temurin:17-jdk-alpine AS build
```

```
WORKDIR /app
```

```
COPY pom.xml .
```

```
COPY src ./src
```

```
RUN ./mvnw package -DskipTests 2>/dev/null || mvn package -DskipTests
```

```
FROM eclipse-temurin:17-jre-alpine
```

```
WORKDIR /app
```

```
COPY --from=build /app/target/tuckit-backend-1.0.0.jar app.jar
```

```
EXPOSE 8080
```

```
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Build and run:

```
# Build the Docker image
```

```
docker build -t tuckit-backend ./backend
```

```
# Run (mount your application.properties as a volume)
```

```
docker run -p 8080:8080 \
```

```
-v $(pwd)/backend/src/main/resources/application.properties:/app/application.properties \
```

```
tuckit-backend
```

Security Best Practices

⚠ Never commit real secrets to Git. If you accidentally commit an API key, immediately revoke it and generate a new one.

Practice	Why
✅ Keep application.properties in .gitignore	Prevents leaking database credentials, API keys
✅ Use Gmail App Passwords, not your real password	Real password gives full account access
✅ Use Razorpay test keys during development	Prevents real charges

Practice	Why
✅ Generate a strong, unique JWT secret (64+ chars)	Prevents token forgery
✅ Rotate API keys regularly	Limits damage if a key is ever exposed
✅ Never log passwords or tokens in the code	Logs may be stored insecurely
✅ Use MongoDB Atlas IP whitelisting in production	Restrict DB access to your server's IP only

Recommended .gitignore

The repo already includes a proper .gitignore. For reference, here are the critical entries:

Secrets — NEVER commit these

backend/src/main/resources/application.properties

*.env

.env

Build output

target/

client/dist/

Node packages

client/node_modules/

IDE files

.idea/

.vscode/

*.iml